

Universidad Americana

Programación de Aplicaciones de Escritorio

Nombre: María Celeste Carrasco Obando

Docente: José Duran García

Lunes 19 de octubre del 2026

1. ¿Qué diferencia existe entre Thread.sleep() y Task.cancel(true)?

- **Thread.sleep()**: Pausa la ejecución del hilo actual durante un tiempo determinado (por ejemplo, 250 ms), liberando temporalmente el uso del procesador pero sin solicitar la detención del hilo.
- **Task.cancel(true)**: Envía una solicitud de cancelación a la tarea. Al pasar true, interrumpe el hilo ejecutando interrupt(), lo que hace que isCancelled() pase a true y fuerza a que métodos bloqueantes como Thread.sleep() lancen una excepción InterruptedException.

2. ¿Por qué no se debe ejecutar el procesamiento directamente dentro de setOnAction?

Porque las acciones definidas en setOnAction se ejecutan en el **JavaFX Application Thread** (el hilo encargado de la interfaz gráfica). Si realizas tareas pesadas o bucles largos en este hilo, la interfaz se **congelará**, dejará de responder a los clics del usuario y no podrá redibujar ni actualizar los componentes visuales en pantalla.

3. ¿Qué protege synchronized y qué problema no resuelve por sí solo?

- **Qué protege**: Garantiza el acceso exclusivo a un método o bloque de código (exclusión mutua), evitando **condiciones de carrera** (*race conditions*) cuando varios hilos modifican o leen un recurso compartido al mismo tiempo.
- **Qué no resuelve por sí solo**: No evita **interbloqueos** (*deadlocks*), **inanición** (*starvation*), ni garantiza el orden en que las tareas se despachan o renderizan en la interfaz gráfica (Platform.runLater()).

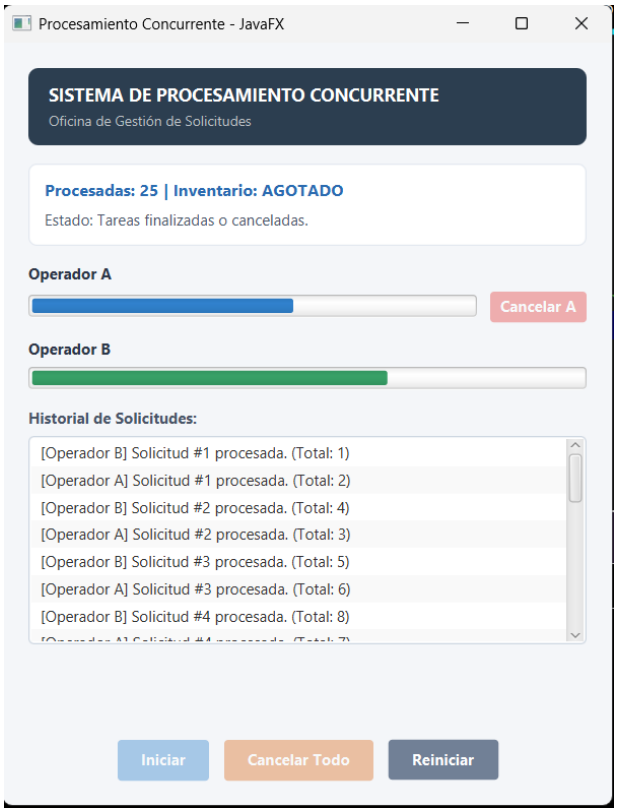
4. ¿Cuándo utilizaría Platform.runLater() y cuándo updateProgress()?

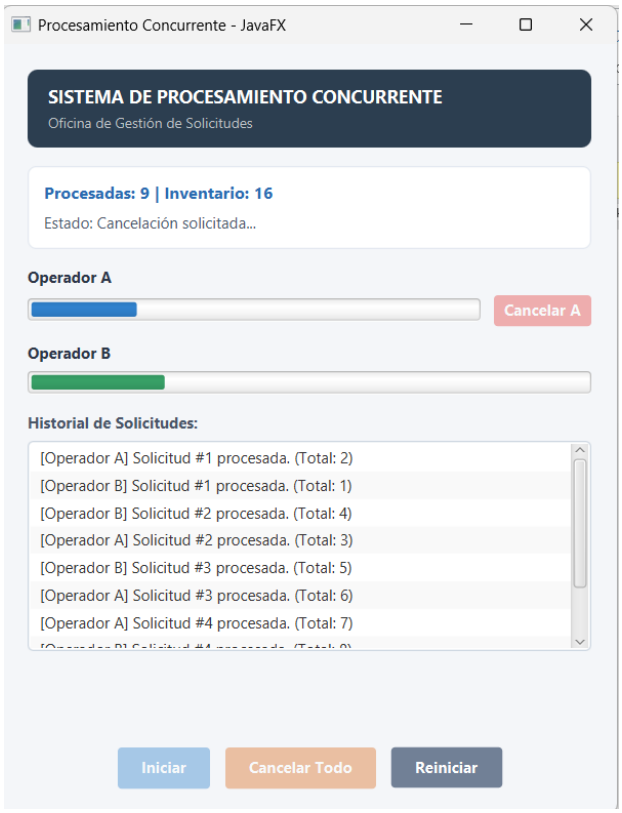
- **updateProgress()**: Se utiliza exclusivamente dentro de una clase Task para actualizar las propiedades de avance vinculadas a un ProgressBar o ProgressIndicator. Es un método diseñado específicamente para ser invocado de forma segura desde un hilo secundario.
- **Platform.runLater()**: Se utiliza para ejecutar código en el hilo de JavaFX cuando se necesita actualizar cualquier otro componente gráfico (Label, ListView, botones, etc.) directamente desde un hilo secundario.

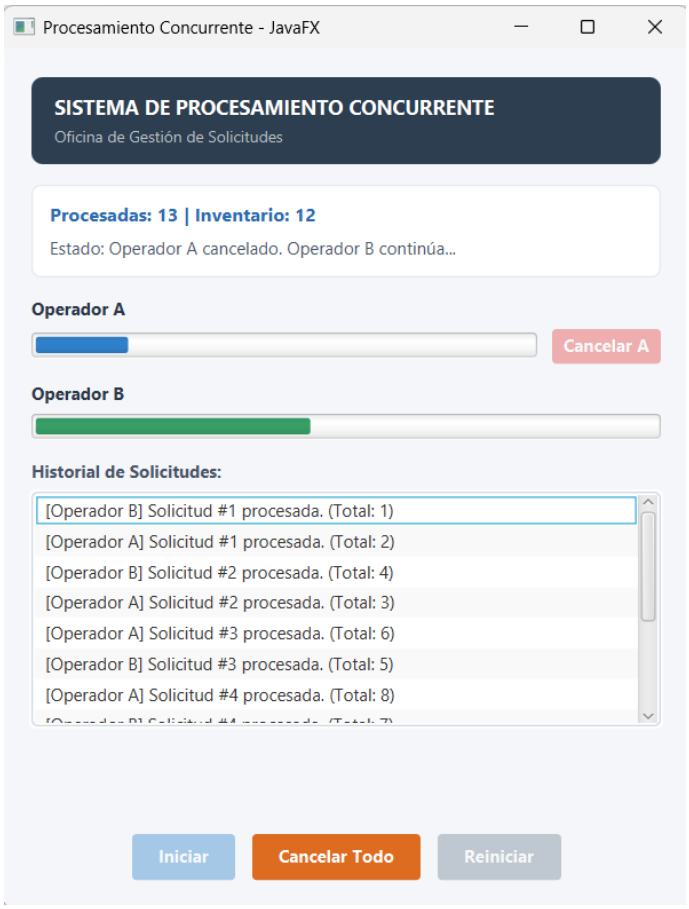
5. ¿Por qué la cancelación de un hilo es cooperativa?

Porque en Java un hilo no debe ser detenido de manera abrupta por un tercero, ya que esto podría dejar archivos abiertos, conexiones colgadas o datos en un estado inconsistente en memoria. La cancelación es **cooperativa** porque la tarea en segundo plano debe **verificar activamente** si le han solicitado cancelar (mediante isCancelled()) y decidir por sí misma limpiar sus recursos y finalizar su ejecución de forma segura.

Capturas de ejecución






Procesamiento Concurrente - JavaFX
SISTEMA DE PROCESAMIENTO CONCURRENTE
Oficina de Gestión de Solicitudes
Procesadas: 13 | Inventario: 12
Estado: Operador A cancelado. Operador B continúa...
Operador A
Cancelar A
Operador B
Historial de Solicitudes:
[Operador B] Solicitud #1 procesada. (Total: 1)
[Operador A] Solicitud #1 procesada. (Total: 2)
[Operador B] Solicitud #2 procesada. (Total: 4)
[Operador A] Solicitud #2 procesada. (Total: 3)
[Operador A] Solicitud #3 procesada. (Total: 6)
[Operador B] Solicitud #3 procesada. (Total: 5)
[Operador A] Solicitud #4 procesada. (Total: 8)
Iniciar
Cancelar Todo
Reiniciar